

Week 3: DAY 13 – Hands On – Keerthivasan R

Pre-Requisites: Before starting with problem solving, please make sure that you have created a database and restored data (refer **Sample Database** create **objects.txt** and **load data.txt**)

Level-1: Problem 1 - Basic Customer Order Report

Scenario:

The store manager wants a simple report showing customer orders along with their order dates and status. This report will help track pending and completed orders.

✦ Requirements

1. Retrieve customer first name, last name, order_id, order_date, and order_status.
2. Display only orders with status Pending (1) or Completed (4).
3. Sort the results by order_date in descending order.

🔧 Technical Constraints

- Use SELECT statement.
- Use WHERE clause with logical operators (AND/OR).
- Use ORDER BY clause.
- Use INNER JOIN between customers and orders tables.

Expectations:

Students should write a correct query using joins and filters, and properly order the result set.

🎯 Learning Outcome

Understand basic SELECT queries, filtering using WHERE conditions, logical operators, and sorting using ORDER BY clause with INNER JOIN.

QUERY:

```
-- Step 1: If database exists, delete it
IF DB_ID('StoreDb') IS NOT NULL
BEGIN
    ALTER DATABASE StoreDb SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
    DROP DATABASE StoreDb;
END
GO

-- Step 2: Create database
CREATE DATABASE StoreDb;
GO
```

```

USE StoreDb;
GO

-- Step 3: Create customers table
CREATE TABLE customers
(
    customer_id INT PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL
);

-- Step 4: Create orders table
CREATE TABLE orders
(
    order_id INT PRIMARY KEY,
    customer_id INT NOT NULL,
    order_date DATETIME NOT NULL,
    order_status INT NOT NULL,

    CONSTRAINT FK_orders_customer
        FOREIGN KEY (customer_id)
        REFERENCES customers(customer_id)
);

-- Step 5: Insert customers
INSERT INTO customers VALUES
(1, 'John', 'Smith'),
(2, 'Emma', 'Watson'),
(3, 'David', 'Miller');

-- Step 6: Insert orders
INSERT INTO orders VALUES
(101, 1, '2026-03-01', 1),
(102, 1, '2026-03-05', 4),
(103, 2, '2026-03-10', 2),
(104, 3, '2026-03-15', 1),
(105, 2, '2026-03-20', 4);

-- Step 7: Final Required Query
SELECT
    c.first_name,
    c.last_name,
    o.order_id,
    o.order_date,
    o.order_status
FROM customers c
INNER JOIN orders o
    ON c.customer_id = o.customer_id
WHERE o.order_status IN (1,4)
ORDER BY o.order_date DESC;

```

OUTPUT:

	first_name	last_name	order_id	order_date	order_status
1	Emma	Watson	105	2026-03-20 00:00:00.000	4
2	David	Miller	104	2026-03-15 00:00:00.000	1
3	John	Smith	102	2026-03-05 00:00:00.000	4
4	John	Smith	101	2026-03-01 00:00:00.000	1

Level-1: Problem 2 - Product Price Listing by Category

Scenario:

The sales team wants a product listing categorized by product category along with brand details to understand product distribution.

Requirements

1. Display product_name, brand_name, category_name, model_year, and list_price.
2. Filter products with list_price greater than 500.
3. Sort results by list_price in ascending order.

Technical Constraints

- Use SELECT statement.
- Use WHERE clause.
- Use ORDER BY clause.
- Use INNER JOIN between products, brands, and categories tables.

Expectations:

Students should correctly join multiple tables and apply filtering and sorting logic.

Learning Outcome

Learn multi-table joins, filtering numeric conditions, and sorting query results.

QUERY:

```
USE StoreDb;
GO

/* =====
   DROP TABLES (Child → Parent Order)
===== */

IF OBJECT_ID('stocks','U') IS NOT NULL DROP TABLE stocks;
IF OBJECT_ID('order_items','U') IS NOT NULL DROP TABLE order_items;
IF OBJECT_ID('products','U') IS NOT NULL DROP TABLE products;
IF OBJECT_ID('brands','U') IS NOT NULL DROP TABLE brands;
IF OBJECT_ID('categories','U') IS NOT NULL DROP TABLE categories;
IF OBJECT_ID('stores','U') IS NOT NULL DROP TABLE stores;
GO

/* =====
   CREATE TABLES
===== */

-- Brands
CREATE TABLE brands (
    brand_id INT PRIMARY KEY,
    brand_name VARCHAR(100)
);

-- Categories
CREATE TABLE categories (
```

```

    category_id INT PRIMARY KEY,
    category_name VARCHAR(100)
);

-- Products
CREATE TABLE products (
    product_id INT PRIMARY KEY,
    product_name VARCHAR(100),
    brand_id INT,
    category_id INT,
    model_year INT,
    list_price DECIMAL(10,2),
    FOREIGN KEY (brand_id) REFERENCES brands(brand_id),
    FOREIGN KEY (category_id) REFERENCES categories(category_id)
);

-- Stores
CREATE TABLE stores (
    store_id INT PRIMARY KEY,
    store_name VARCHAR(100)
);

-- Order Items
CREATE TABLE order_items (
    order_item_id INT PRIMARY KEY,
    order_id INT,
    product_id INT,
    quantity INT,
    list_price DECIMAL(10,2),
    discount DECIMAL(4,2),
    FOREIGN KEY (order_id) REFERENCES orders(order_id),
    FOREIGN KEY (product_id) REFERENCES products(product_id)
);

-- Stocks
CREATE TABLE stocks (
    store_id INT,
    product_id INT,
    quantity INT,
    PRIMARY KEY (store_id, product_id),
    FOREIGN KEY (store_id) REFERENCES stores(store_id),
    FOREIGN KEY (product_id) REFERENCES products(product_id)
);

GO

/* =====
   INSERT DATA
   ===== */

INSERT INTO brands VALUES
(1, 'Nike'),
(2, 'Adidas');

INSERT INTO categories VALUES
(1, 'Shoes'),
(2, 'Accessories');

INSERT INTO products VALUES
(1, 'Air Max', 1, 1, 2024, 800),
(2, 'Running Pro', 2, 1, 2023, 600),
(3, 'Cap Classic', 1, 2, 2024, 200);

INSERT INTO stores VALUES
(1, 'Central Store'),

```

```

(2, 'City Mall Store');

INSERT INTO order_items VALUES
(1,102,1,2,800,0.10),
(2,105,2,1,600,0.05);

INSERT INTO stocks VALUES
(1,1,20),
(1,2,15),
(2,1,10),
(2,3,50);

GO

/* =====
   FINAL QUERY (Level-1 Problem-2)
   ===== */

SELECT
    p.product_name,
    b.brand_name,
    c.category_name,
    p.model_year,
    p.list_price
FROM products p
INNER JOIN brands b
    ON p.brand_id = b.brand_id
INNER JOIN categories c
    ON p.category_id = c.category_id
WHERE p.list_price > 500
ORDER BY p.list_price ASC;

```

OUTPUT:

	product_name	brand_name	category_name	model_year	list_price
1	Running Pro	Adidas	Shoes	2023	600.00
2	Air Max	Nike	Shoes	2024	800.00

Level-2: Problem 1 - Store Wise Sales Summary

Scenario:

The regional manager wants to analyze total sales generated by each store.

Requirements

1. Display store_name and total sales amount.
2. Calculate total sales using (quantity * list_price * (1 - discount)).
3. Include only completed orders (order_status = 4).
4. Group results by store_name.
5. Sort total sales in descending order.

Technical Constraints

- Use INNER JOIN between stores, orders, and order_items tables.
- Use GROUP BY clause.
- Use aggregate functions such as SUM().
- Use WHERE clause for filtering.
- Use ORDER BY clause.

Expectations:

Students must write a grouped query with calculated columns and apply proper filtering.

Learning Outcome

Understand GROUP BY, aggregate functions, calculated fields, and filtering grouped data.

QUERY:

```
USE StoreDb;
GO

/* =====
STEP 1: Modify Orders Table
===== */

-- Drop order_items first (child table)
IF OBJECT_ID('order_items', 'U') IS NOT NULL DROP TABLE order_items;
GO

-- Drop and recreate orders with store_id
IF OBJECT_ID('orders', 'U') IS NOT NULL DROP TABLE orders;
GO

CREATE TABLE orders
(
    order_id INT PRIMARY KEY,
    customer_id INT NOT NULL,
    store_id INT NOT NULL,
    order_date DATETIME NOT NULL,
```

```

        order_status INT NOT NULL,
        FOREIGN KEY (customer_id) REFERENCES customers(customer_id),
        FOREIGN KEY (store_id) REFERENCES stores(store_id)
    );
GO

```

```

/* =====
STEP 2: Recreate order_items
===== */

```

```

CREATE TABLE order_items (
    order_item_id INT PRIMARY KEY,
    order_id INT,
    product_id INT,
    quantity INT,
    list_price DECIMAL(10,2),
    discount DECIMAL(4,2),
    FOREIGN KEY (order_id) REFERENCES orders(order_id),
    FOREIGN KEY (product_id) REFERENCES products(product_id)
);
GO

```

```

/* =====
STEP 3: Insert Orders Data
===== */

```

```

INSERT INTO orders VALUES
(101, 1, 1, '2026-03-01', 1),
(102, 1, 1, '2026-03-05', 4),
(103, 2, 2, '2026-03-10', 4),
(104, 3, 2, '2026-03-15', 1),
(105, 2, 1, '2026-03-20', 4);

```

```

INSERT INTO order_items VALUES
(1, 102, 1, 2, 800, 0.10),
(2, 105, 2, 1, 600, 0.05),
(3, 103, 1, 1, 800, 0.00);
GO

```

```

/* =====
STEP 4: LEVEL-2 PROBLEM-1 QUERY
===== */

```

```

SELECT
    s.store_name,
    SUM(oi.quantity * oi.list_price * (1 - oi.discount)) AS total_sales
FROM stores s
INNER JOIN orders o
    ON s.store_id = o.store_id
INNER JOIN order_items oi
    ON o.order_id = oi.order_id
WHERE o.order_status = 4
GROUP BY s.store_name
ORDER BY total_sales DESC;

```

OUTPUT:

	store_name	total_sales
1	Central Store	2010.0000
2	City Mall Store	800.0000

Level-2: Problem 2 - Product Stock and Sales Analysis

Scenario:

The inventory manager wants to compare stock availability and total quantity sold for each product.

Requirements

1. Display product_name, store_name, available stock quantity, and total quantity sold.
2. Include products even if they have not been sold (use appropriate join).
3. Group results by product_name and store_name.
4. Sort results by product_name.

Technical Constraints

- Use LEFT JOIN between stocks and order_items.
- Use INNER JOIN with products and stores tables.
- Use GROUP BY clause.
- Use SUM() aggregate function.
- Use ORDER BY clause.

Expectations:

Students should correctly use different join types (INNER and LEFT), grouping, and aggregate functions.

Learning Outcome

Develop strong understanding of different JOIN types (INNER, LEFT), aggregation, grouping data, and analyzing relational datasets.

QUERY:

```
USE StoreDb;
GO

SELECT
    p.product_name,
    s.store_name,
    st.quantity AS available_stock,
    ISNULL(SUM(oi.quantity), 0) AS total_quantity_sold
FROM stocks st

-- INNER JOIN products
INNER JOIN products p
    ON st.product_id = p.product_id
```

```

-- INNER JOIN stores
INNER JOIN stores s
    ON st.store_id = s.store_id

-- LEFT JOIN orders (only completed orders optional if needed)
LEFT JOIN orders o
    ON st.store_id = o.store_id
    AND o.order_status = 4

-- LEFT JOIN order_items
LEFT JOIN order_items oi
    ON o.order_id = oi.order_id
    AND st.product_id = oi.product_id

GROUP BY
    p.product_name,
    s.store_name,
    st.quantity

ORDER BY
    p.product_name ASC;

```

OUTPUT:

	product_name	store_name	available_stock	total_quantity_sold
1	Air Max	Central Store	20	2
2	Air Max	City Mall Store	10	1
3	Cap Classic	City Mall Store	50	0
4	Running Pro	Central Store	15	1